Name: WAQAR RIASAT ALI Section: E Roll num: 2023F - BSE - 221

LAB NO 2

DATA STRUCTURES AND ALGORITHMS

OBJECTIVE: To implement ArrayList and Vector.

LAB TASKS

1. Write a program that initializes Vector with 10 integers in it. Display all the integers and sum of these integers.

Input 01:

```
    Vector<Integer> my_first_vector = new Vector<> (10);

    my_first_vector.add(20);
    my_first_vector.add(19);
    my_first_vector.add(18);
    my_first_vector.add(17);
    my_first_vector.add(16);
    my_first_vector.add(15);
    my_first_vector.add(14);
    my_first_vector.add(13);
    my_first_vector.add(12);
    my_first_vector.add(11);


    int sum = 0;
        for(int num : my_first_vector){
            sum += num;
        }

    System.out.println( "the added  numbers were:" + my_first_vector + " \n and the sum of the vector is : " + sum );
    }
}
```

Output 01:

```
Output - LAB NO 2 (run)
run:
the added  numbers were:[20, 19, 18, 17, 16, 15, 14, 13, 12, 11]
 and the sum of the vector is : 155
BUILD SUCCESSFUL (total time: 0 seconds)
```

2. Create a ArrayList of string. Write a menu driven program which: a. Displays all the elements b. Displays the largest String

Input 02:

```
    ArrayList < String > a1 = new ArrayList<> ();
    a1.add("Halwa puri ");
    a1.add("Khatri ki biryani");
    a1.add("Lala ki chai");
    a1.add("Shams ki chartt");
    a1.add("munir ke samose");
    a1.add("Fresco ki Jaleebi");

    System.out.println("displaying the menue :\n "+a1);

    String largest = "";
    for(String item : a1){

        if(item.length() >  largest.length()){
            largest = item;
        }

    }
        System.out.println();
        System.out.println("The largest String is: " + largest);

    }
}
```

Output 02:

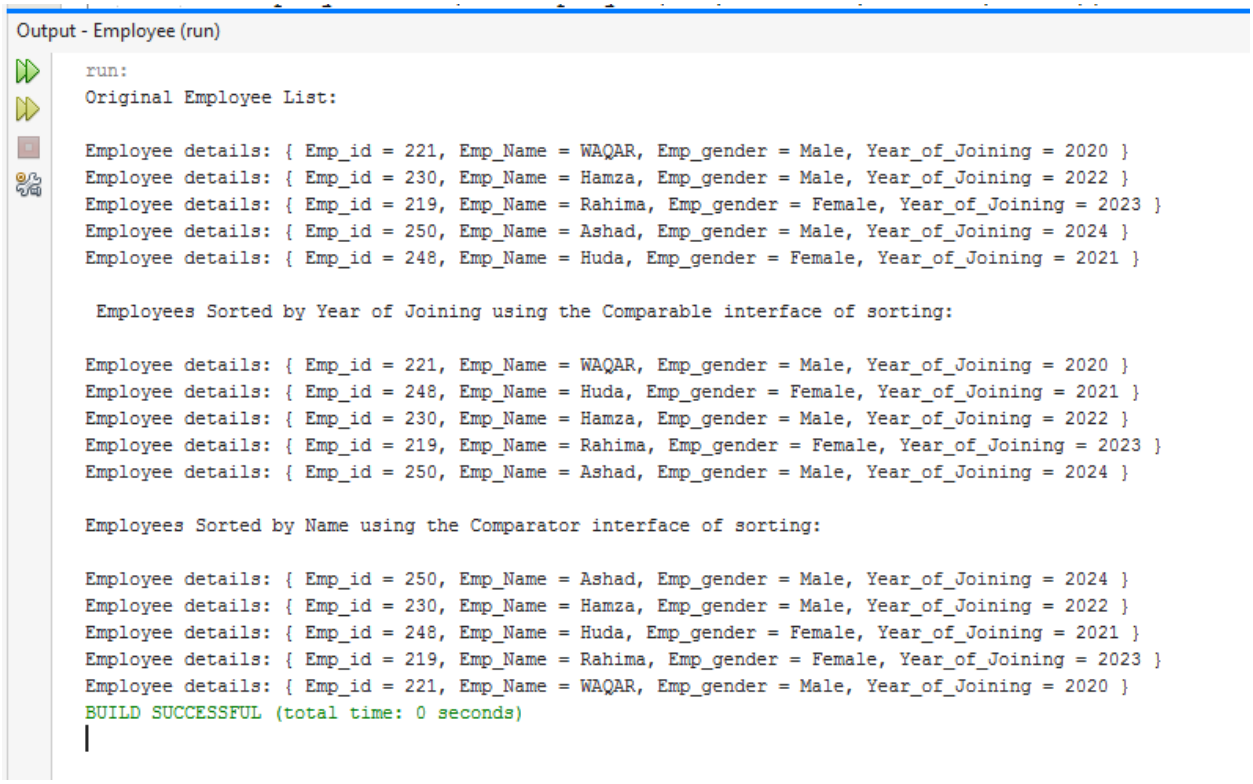
```
Output - LAB NO 2 (run)
run:
displaying the menue :
 [Halwa puri , Khatri ki biryani, Lala ki chai, Shams ki chartt, munir ke samose, Fresco ki Jaleebi]

The largest String is: Khatri ki biryani
BUILD SUCCESSFUL (total time: 0 seconds)
```

3. Create a Arraylist storing Employee details including Emp_id, Emp_Name, Emp_gender, Year_of_Joining (you can also add more attributes including these). Then sort the employees according to their joining year using Comparator and Comparable interfaces.

Input 03:

```
//     using the toString method to display details of each employee
    public String toString() {
        return "Employee details: { Emp_id = " + emp_idi + ", Emp_Name = " + emp_Name +
                ", Emp_gender = " + emp_Gender + ", Year_of_Joining = " + year_Of_Joining + " }";
    }

 // method of comparable interface  for natural ordering by year of joining

    public int compareTo(Employee other) {
        return Integer.compare(this.year_Of_Joining, other.year_Of_Joining);
    }
}

// Comparator for sorting by name
class NameComparator implements Comparator<Employee> {

    public int compare(Employee e1, Employee e2) {
        return e1.getEmpName().compareTo(e2.getEmpName());
    }
}

class Sort_Employee {
    public static void main(String[] args) {
        ArrayList<Employee> employees = new ArrayList<>();
        employees.add(new Employee(221, "WAQAR", "Male", 2020));
        employees.add(new Employee(230, "Hamza", "Male", 2022));
        employees.add(new Employee(219, "Rahima", "Female", 2023));
        employees.add(new Employee(250, "Ashad", "Male", 2024));
        employees.add(new Employee(248, "Huda", "Female", 2021));

        System.out.println("Original Employee List:");
        System.out.println();
        for (Employee emp : employees) {
            System.out.println(emp);
        }

        // Sort employees by year of joining using Comparable
        Collections.sort(employees);
        System.out.println("\n Employees Sorted by Year of Joining using the Comparable interface of sorting:");
        System.out.println();
        for (Employee emp : employees) {
            System.out.println(emp);
        }
        // Sort employees by name using Comparator
        Collections.sort(employees, new NameComparator());
        System.out.println("\nEmployees Sorted by Name using the Comparator interface of sorting:");
        System.out.println();
        for (Employee emp : employees) {
            System.out.println(emp);
```

Output 03:



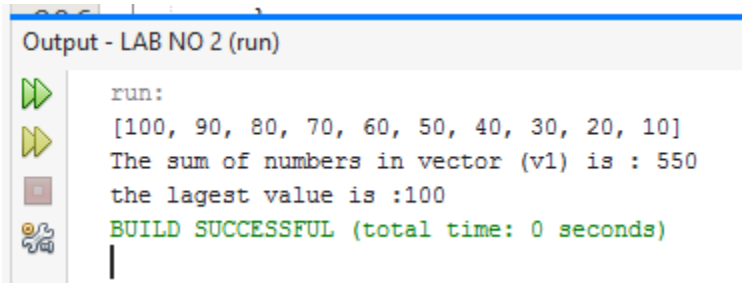
4. Write a program that initializes Vector with 10 integers in it.
   - Display all the integers
   - Sum of these integers.
   - Find Maximum Element in Vector

Input 04:

```
        Vector <Integer> v1 = new Vector <> (10);

        v1.add(100);
        v1.add(90);
        v1.add(80);
        v1.add(70);
        v1.add(60);
        v1.add(50);
        v1.add(40);
        v1.add(30);
        v1.add(20);
        v1.add(10);

//      part A
        System.out.println(v1);

//      part B
    int sum = 0;
        for(int num : v1){
            sum+= num;
        }
    System.out.println("The sum of numbers in vector (v1) is : "+ sum);
```

```
//      part C
    int max = 0;
    for(int num : v1){

        if(num > max){
            max = num;
        }
    }
    System.out.println("the lagest value is :" + max);
    }
}
```

Output 04:

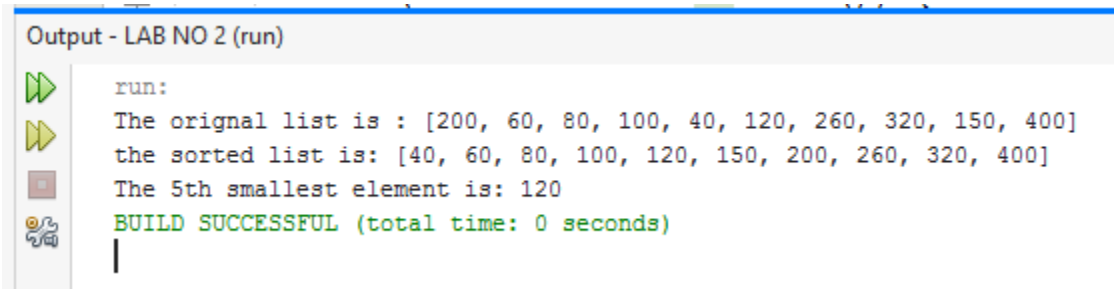


5. Find the k-th smallest element in a sorted ArrayList

Input 05:

```
    Vector <Integer> v1 = new Vector <> (10);
334     v1.add(200);
335     v1.add(60);
336     v1.add(80);
337     v1.add(100);
338     v1.add(40);
339     v1.add(120);
340     v1.add(260);
341     v1.add(320);
342     v1.add(150);
343     v1.add(400);
344
345     System.out.println("The orignal list is : " + v1);
346     Collections.sort(v1);
347     System.out.println("the sorted list is: " + v1);
348
349         int k = 3;
350         if (k > 0 && k <= v1.size()) {
351             int desired_smallest = v1.get(k - 1);
352             System.out.println("The " + k + "th smallest element is: " + desired_smallest);
353         }
354         else {
355             System.out.println("Invalid value of k is entered \n please entre the correct value...");}}}
```

Output 05:



6. Write a program to merge two ArrayLists into one.

Input 06:

```
        ArrayList <Integer> a1 = new ArrayList <> (10);
            a1.add(110);
            a1.add(100);
            a1.add(90);
            a1.add(80);
            a1.add(70);
            a1.add(60);
            a1.add(50);
            a1.add(40);
            a1.add(30);
            a1.add(20);

            ArrayList <Integer> a2 = new ArrayList <> (10);
            a2.add(10);
            a2.add(9);
            a2.add(8);
            a2.add(7);
            a2.add(6);
            a2.add(5);
            a2.add(4);
            a2.add(3);
            a2.add(2);
            a2.add(1);

            System.out.println("The list a2 is: " + a2);
            ArrayList<Integer> a3 = new ArrayList<>(a1.size() + a2.size());

            a3.addAll(a1);
            a3.addAll(a2);
            System.out.println();
            System.out.println("The merged list a3 is: " + a3);
    }
}
```

Output 06:

```
Output - LAB NO 2 (run)
run:
The list a1 is: [110, 100, 90, 80, 70, 60, 50, 40, 30, 20]
The list a2 is: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

The merged list a3 is: [110, 100, 90, 80, 70, 60, 50, 40, 30, 20, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
BUILD SUCCESSFUL (total time: 0 seconds)
```

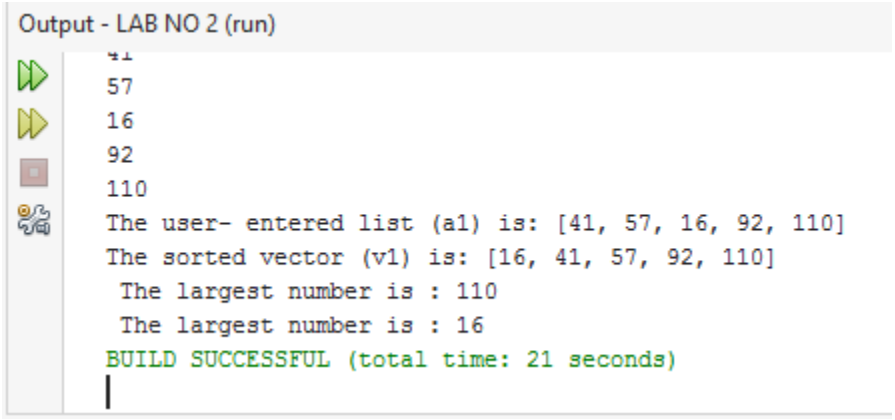
HOME TASKS

1. Create a Vector storing integer objects as an input.
   a. Sort the vector
   b. Display largest number
   c. Display smallest number

Input 01:

```
        Vector <Integer> v1 = new Vector <>();

        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the size of vector");
        int n = sc.nextInt();

         System.out.println("Enter integers:");
        for (int i = 0; i < n; i++) {
            int input = sc.nextInt();
             v1.add(input);
        }
        System.out.println("The user- entered list (a1) is: " + v1);

//          PART A
        Collections.sort(v1);
        System.out.println("The sorted vector (v1) is: " + v1);
//          PART B
         System.out.println(" The largest number is : " + v1.lastElement());
//          PART C
         System.out.println(" The largest number is : " + v1.firstElement());
```

Output 01:

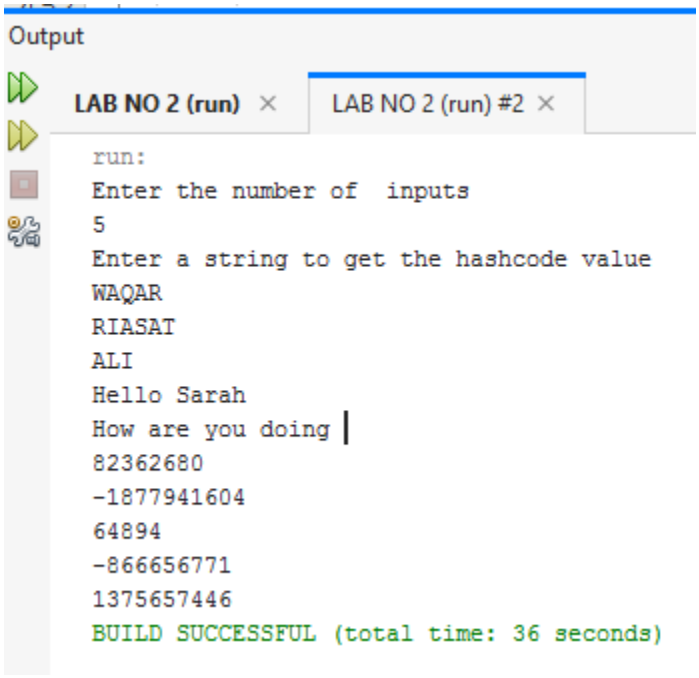


2. Write a java program which takes user input and gives hashcode value of those inputs using hashCode () method.

Input 02:

```
//  Write a java program which takes user input and gives hashcode value of those inputs using hashCode () method
    public static void main(String[] args) {

       Vector <String> v1 = new Vector <>();

       Scanner sc = new Scanner(System.in);
       System.out.println("Enter the number of  inputs");
       int n = sc.nextInt();

       sc.nextLine();

        System.out.println("Enter a string to get the hashcode value");

        for(int i =0 ; i<n ; i++){
        String input = sc.nextLine();
        v1.add(input);
        }
        for(String words : v1){
         System.out.println(words.hashCode());
        }
```

Output 02:



3. Scenario based

Create a java project, suppose you work for a company that needs to manage a list of employees. Each employee has a unique combination of a name and an ID. Your goal is to ensure that you can track employees effectively and avoid duplicate entries in your system.

Requirements:

a. Employee Class: You need to create an Employee class that includes:

- name: The employee's name (String).
- id: The employee's unique identifier (int).
- Override the hashCode() and equals() methods to ensure that two employees are considered equal if they have the same name and id.

b. Employee Management: You will use a HashSet to store employee records. This will help you avoid duplicate entries.

c. Operations: Implement operations to:

- Add new employees to the record.
- Check if an employee already exists in the records.
- Display all employees.

Name: WAQAR RIASAT ALI Section: E Roll num: 2023F - BSE - 221

Input 03:

```java
public class Employee {
    String name;
    int id;

    public Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }

    public int hashCode() {            //  Objects.hash() takes one or more objects (in this case, name and id)
         return Objects.hash(name, id);      //  and generates a single integer hash code from them

    }

    public boolean equals(Object obj) {   // this refers to the current object of the class "Employee"
        if (this == obj)            //  obj refers to the the parameter passed in the equals method..
            return true;   // class ka object or method me pass kiya howa object if same or not
        if (obj == null || getClass() != obj.getClass()) return false;
        Employee employee = (Employee) obj;
        return id == employee.id && Objects.equals(name, employee.name);
    }
```

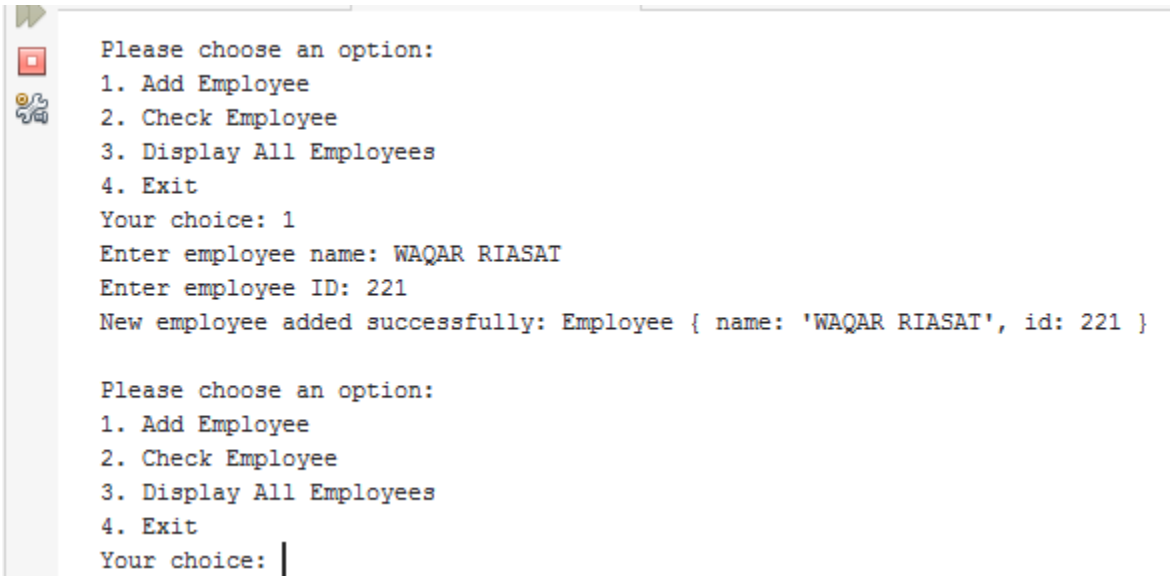
```java
class EmployeeManagement {
    private HashSet<Employee> employeeSet;

    public EmployeeManagement() {
        employeeSet = new HashSet<>();
    }

    public boolean addEmployee(Employee employee) {
        if (employeeSet.contains(employee)) {
            System.out.println("Oops! That employee is already in the records: " + employee);
            return false;
        } else {
            employeeSet.add(employee);
            System.out.println("New employee added successfully: " + employee);
            return true;
        }
    }

    public boolean checkEmployee(Employee employee) {
        return employeeSet.contains(employee);
    }
```

Name: WAQAR RIASAT ALI Section: E Roll num: 2023F - BSE - 221

```
    public boolean checkEmployee(Employee employee) {
        return employeeSet.contains(employee);
    }

    public void displayAllEmployees() {
        if (employeeSet.isEmpty()) {
            System.out.println("No employees found in the system.");
        } else {
            System.out.println("Listing all employees:");
            for (Employee employee : employeeSet) {
                System.out.println(" - " + employee);
            }
        }
    }

    public static void main(String[] args) {
        EmployeeManagement management = new EmployeeManagement();
        Scanner scanner = new Scanner(System.in);

        while (true) {
            System.out.println("\nPlease choose an option:");
            System.out.println("1. Add Employee\n2. Check Employee\n3. Display All Employees\n4. Exit");
            System.out.print("Your choice: ");
            int choice = scanner.nextInt();
            scanner.nextLine();

            if (choice == 4) {
                System.out.println("Exiting the program. Have a great day!");
                break;
            }

            switch (choice) {
                case 1:
                    System.out.print("Enter employee name: ");
                    String name = scanner.nextLine();
                    System.out.print("Enter employee ID: ");
                    int id = scanner.nextInt();
                    Employee employee = new Employee(name, id);
                    management.addEmployee(employee);
                    break;

                case 2:
                    System.out.print("Enter employee name to check: ");
                    name = scanner.nextLine();
                    System.out.print("Enter employee ID to check: ");
                    id = scanner.nextInt();
```

```
                    employee = new Employee(name, id);
                    if (management.checkEmployee(employee)) {
                        System.out.println("Yes, this employee is already in the system: " + employee);
                    } else {
                        System.out.println("Employee not found.");
                    }
                    break;

                case 3:
                    management.displayAllEmployees();
                    break;

                default:
                    System.out.println("Invalid option, please try again.");
            }
        }
        scanner.close();
    }
}
```

Output 03:



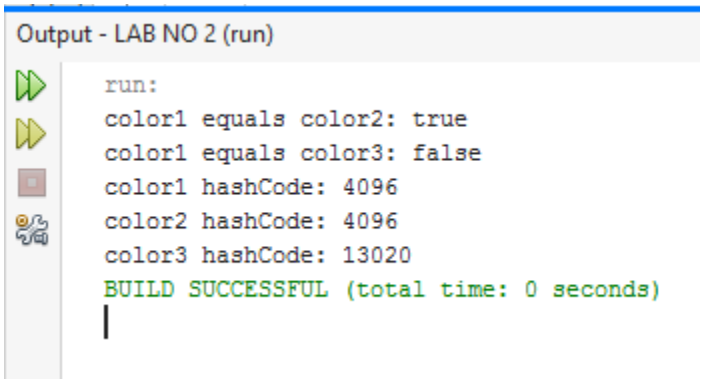
```
Please choose an option:
1. Add Employee
2. Check Employee
3. Display All Employees
4. Exit
Your choice: 1
Enter employee name: WAQAR RIASAT
Enter employee ID: 221
New employee added successfully: Employee { name: 'WAQAR RIASAT', id: 221 }

Please choose an option:
1. Add Employee
2. Check Employee
3. Display All Employees
4. Exit
Your choice:
```

4. Create a Color class that has red, green, and blue values. Two colors are considered equal if their RGB values are the same

Input 04:

```
    public Color(int red, int green, int blue) {
        this.red = red;
        this.green = green;
        this.blue = blue;
    }

    public boolean equals(Object obj) {
        if (this == obj)
            return true;

        if (!(obj instanceof Color))
            return false;

        Color color = (Color) obj;
        return red == color.red && green == color.green && blue == color.blue;
    }

    public int hashCode() {
        return red * 31 + green * 31 + blue;
    }

    public static void main(String[] args) {

        Color color1 = new Color(128, 0, 128);
        Color color2 = new Color(128, 0, 128);
        Color color3 = new Color(255, 165, 0);

        System.out.println("color1 equals color2: " + color1.equals(color2));
        System.out.println("color1 equals color3: " + color1.equals(color3));

        System.out.println("color1 hashCode: " + color1.hashCode());
        System.out.println("color2 hashCode: " + color2.hashCode());
        System.out.println("color3 hashCode: " + color3.hashCode());

    }
}
```

Output 04:


Output - LAB NO 2 (run)
run:
color1 equals color2: true
color1 equals color3: false
color1 hashCode: 4096
color2 hashCode: 4096
color3 hashCode: 13020
BUILD SUCCESSFUL (total time: 0 seconds)